Zenith Core

5-Stage Pipelined RV32I CPU — Verification & Performance Report

Data Hazard Forwarding: Impact on Instruction-Level Timing

This report summarizes functional and performance testbench results for the Zenith Core, a hand-built 5-stage pipelined RV32I core with a UART-driven BIOS, synthesized and tested on FPGA. Two microarchitectural configurations were exercised against an instruction-level test suite: a baseline pipelined datapath with **no data-hazard forwarding** (hazards resolved purely by stalling), and an enhanced datapath with **EX/MEM and MEM/WB forwarding paths**. Every test program below passed functional correctness (RESULT: PASS) in both configurations; this document focuses on the microarchitectural performance delta between them.

Forwarding vs No-Forwarding

40	31%	28%	0
Passing Test Programs	Avg. Steady-State CPI Reduction	Avg. Total-Cycle Reduction	Stall Cycles Remaining (from 316 baseline)

*Key terms: **Steady-State CPI** is cycles-per-instruction with the fixed pipeline fill/drain overhead removed. **Stall cycles** are bubbles inserted by the hazard-detection unit; in the no-forwarding core these are entirely structural/ALU-hazard stalls (register values not yet written back). **Branch flush cycles** come from control hazards and are independent of forwarding, so they appear identically in both configurations.*

Test Methodology

Each assembly test program was assembled and loaded into instruction memory, then run to completion on a cycle-accurate testbench until the core asserted a safe STOP signal. The testbench logged the final register file and data-memory state for functional checking, plus the following microarchitectural counters, which are the basis for every chart and table in this report:

Metric	Description
Total Clock Cycles	Wall-clock cycle count from reset to the STOP signal.
Instructions Retired	Count of instructions that completed the write-back stage.
Measured CPI	Total Clock Cycles / Instructions Retired (includes pipeline fill/drain).
Steady-State CPI	CPI with fixed pipeline fill/drain overhead subtracted out — the fairest apples-to-apples comparison between configurations.
Total Stall Cycles	Bubble cycles inserted by the hazard-detection/stall unit.
Load-Use Stalls	Subset of stalls caused specifically by a load followed immediately by a dependent instruction.
Structural/ALU Stalls	Subset of stalls caused by back-to-back ALU-result dependencies.
Branch Flush Cycles	Pipeline flushes from mispredicted / resolved control-flow instructions.
Simulation Frequency	Host simulator throughput (Hz) — a testbench performance metric, not a property of the RTL core itself.

Two Configurations Compared

No Forwarding — the baseline 5-stage pipeline (IF / ID / EX / MEM / WB) resolves every read-after-write data hazard by stalling the pipeline until the producing instruction has written its result back to the register file. **Forwarding** — the same pipeline augmented with EX/MEM and MEM/WB forwarding paths, routing freshly computed results directly into the EX stage of a dependent instruction and eliminating the great majority of stall cycles. Control-flow (branch/jump) flush cycles are unaffected by forwarding and appear identically in both configurations, since they are resolved later in the pipeline and are a separate hazard class.

Of the test suite, 40 programs produced a full, comparable PASS / PASS (FWD) result pair and are included in every table and chart below. A handful of auxiliary tests (csr_op, exceptions, helloworld, instret, interrupts) reported N/A against these counters and are excluded, as are any individual runs that failed functional checking.

Overall Performance Summary

Averaged across all 40 passing test programs, enabling forwarding cuts steady-state CPI roughly in half relative to the no-forwarding baseline, and removes stalling almost entirely except where a true load-use hazard would otherwise still require a bubble.

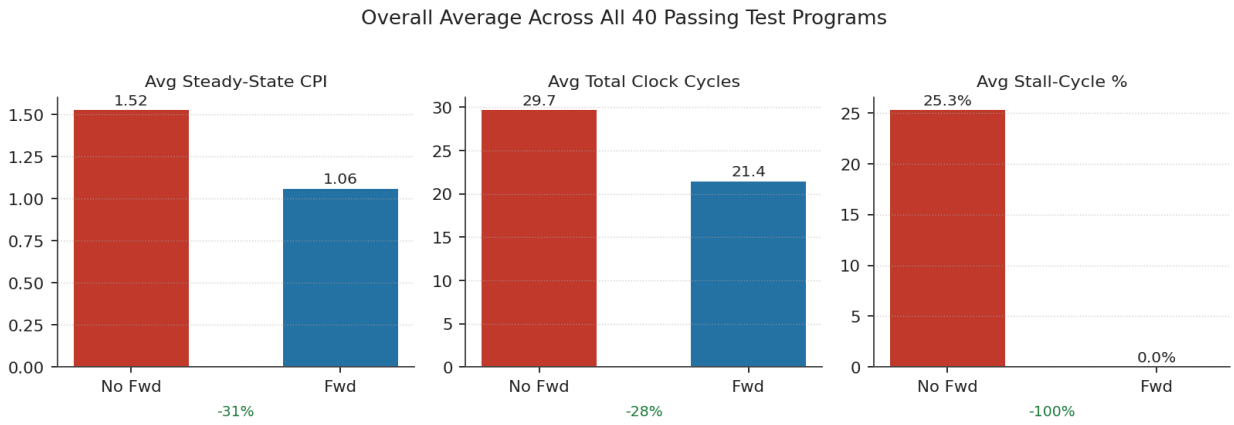


Figure 1. Suite-wide averages: steady-state CPI, total clock cycles, and stall-cycle percentage, no-forwarding vs. forwarding.

Stall Cycles Eliminated

Across the suite, the no-forwarding core accumulates 316 total stall cycles; the forwarding core reduces this to 0 (residual load-use stalls on a small number of load-dependent tests). All logged stalls in the baseline core are structural/ALU-hazard stalls — no load-use-specific stalls were separately logged in this run, meaning the stall unit treats every RAW register dependency uniformly until forwarding is introduced.

CPI Trend Across All Tests

The line charts below plot steady-state CPI for every one of the 40 test programs in sequence, no-forwarding against forwarding. The forwarding line sits flat near 1.0 CPI across almost the entire suite (the theoretical ideal for a single-issue pipeline with no hazards), while the no-forwarding line tracks well above it everywhere a real dependency exists — the gap between the two lines is the cost that forwarding hardware buys back.

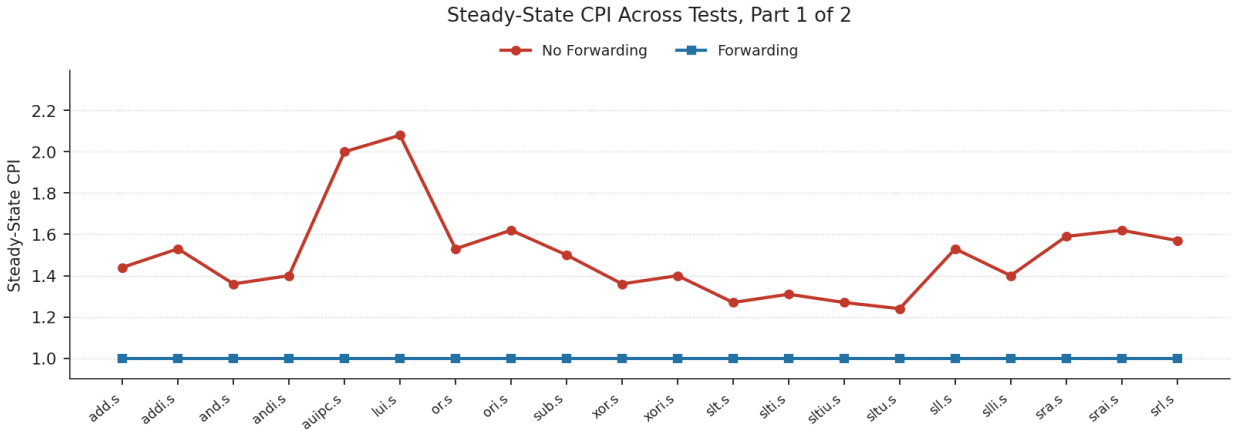


Figure 2a. Steady-state CPI, part 1 of 2 (ALU/logic and shift instructions).

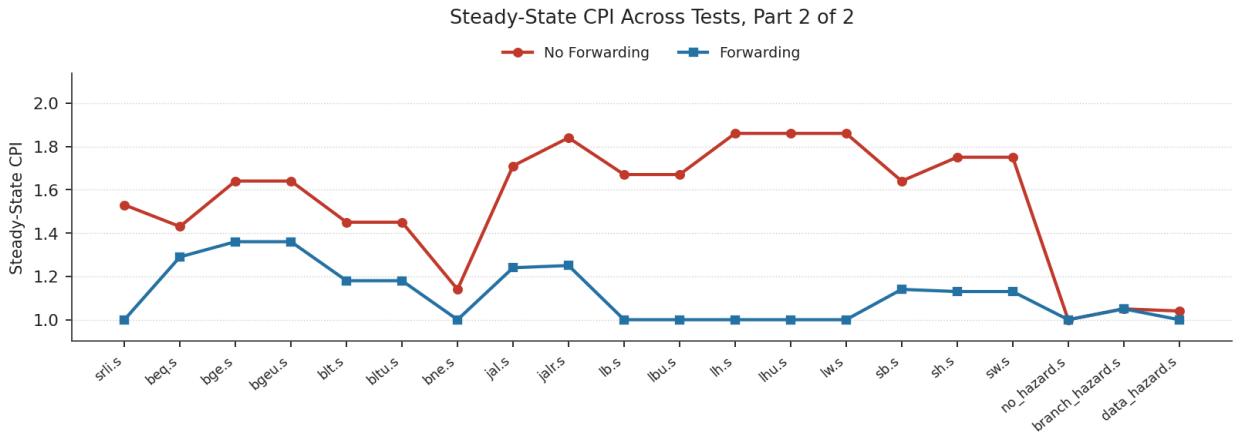


Figure 2b. Steady-state CPI, part 2 of 2 (branches, jumps, loads, stores, hazard-suite tests).

Per-Test Improvement, Ranked

The two charts below rank every test by how much forwarding improved it. Instructions that depend heavily on an immediately preceding ALU result (shifts, LUI/AUIPC chains, back-to-back loads feeding arithmetic) see the largest gains, since those are exactly the dependency patterns forwarding is designed to eliminate. Branches benefit less because part of their cost is branch-resolution flush overhead, which forwarding does not address.

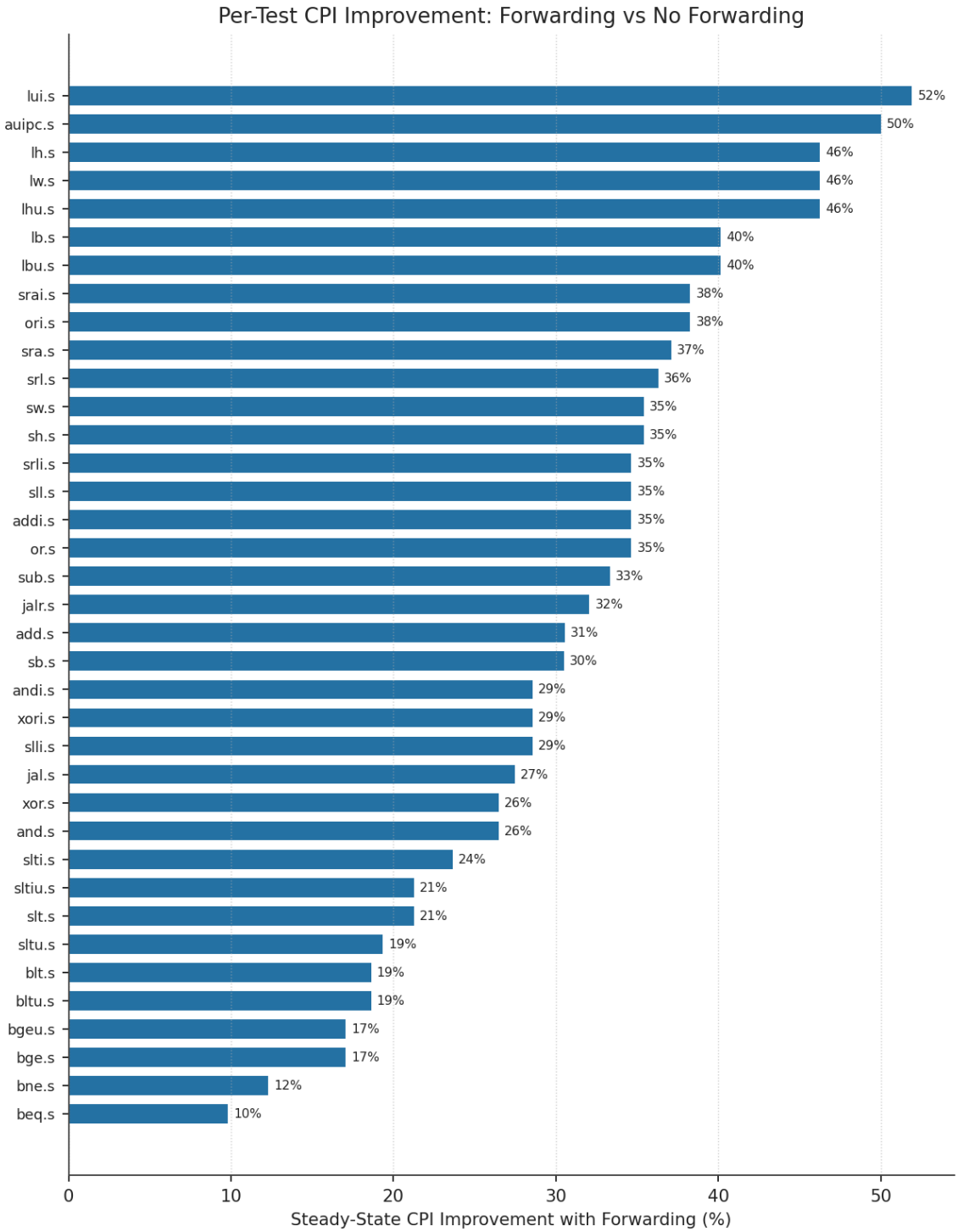


Figure 3. Percentage reduction in steady-state CPI per test program, forwarding vs. no forwarding.

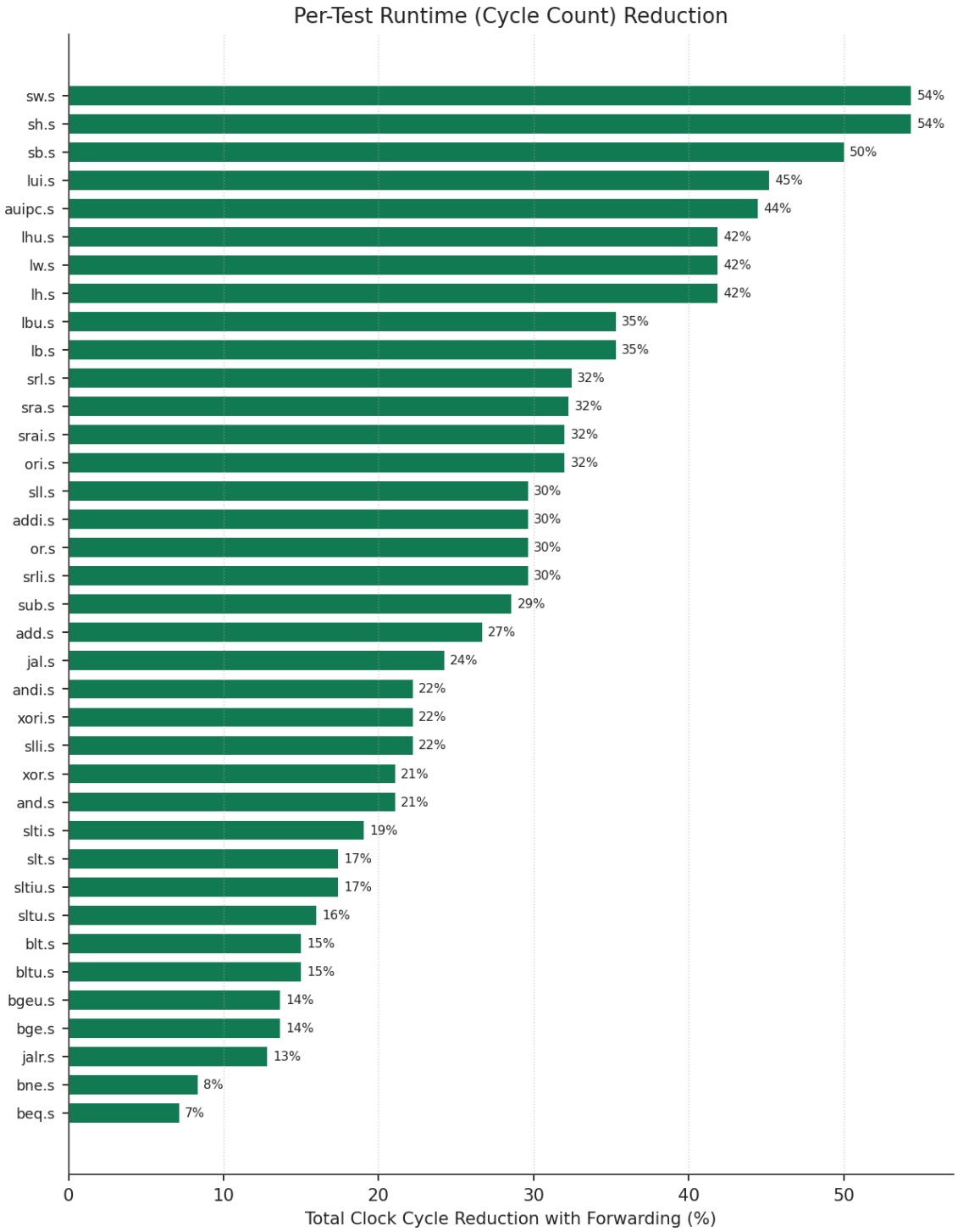
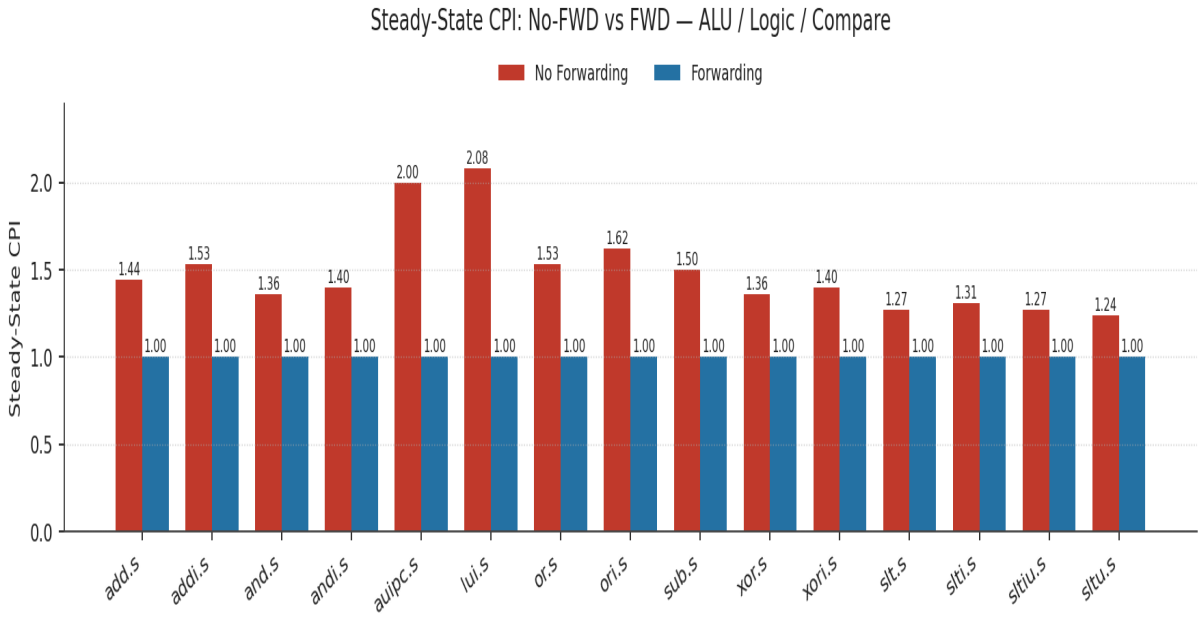


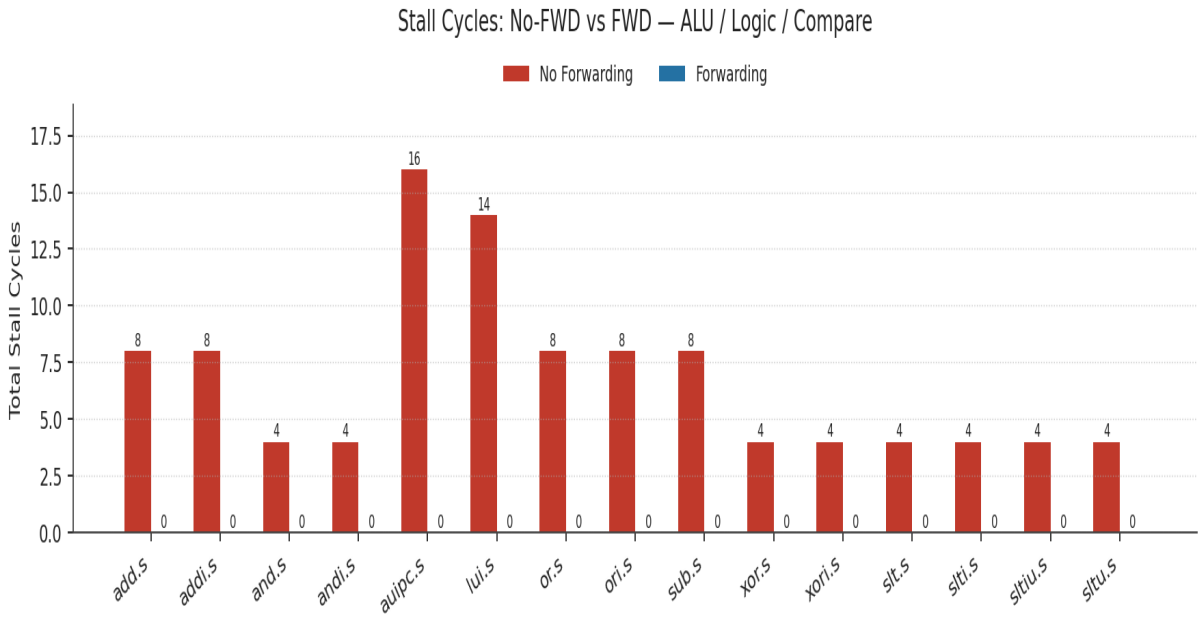
Figure 4. Percentage reduction in total clock cycles to completion per test program.

Breakdown by Instruction Category

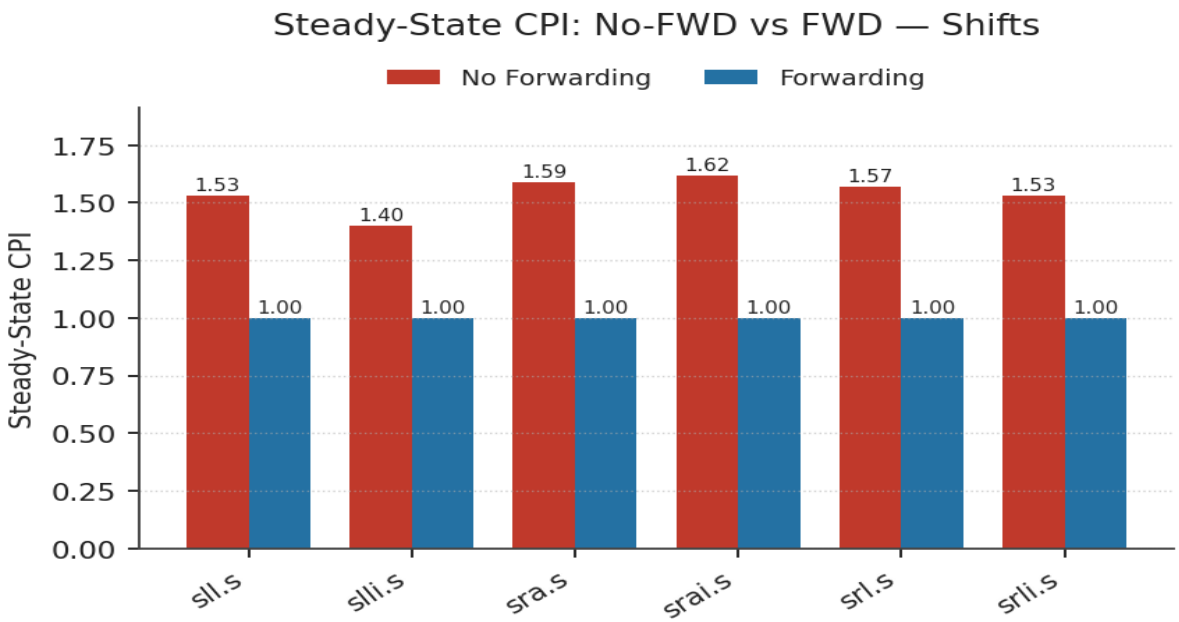
The following pages group tests by instruction type (ALU/logic, shifts, branches, jumps, loads, stores) and show steady-state CPI and stall-cycle counts side-by-side for both configurations.

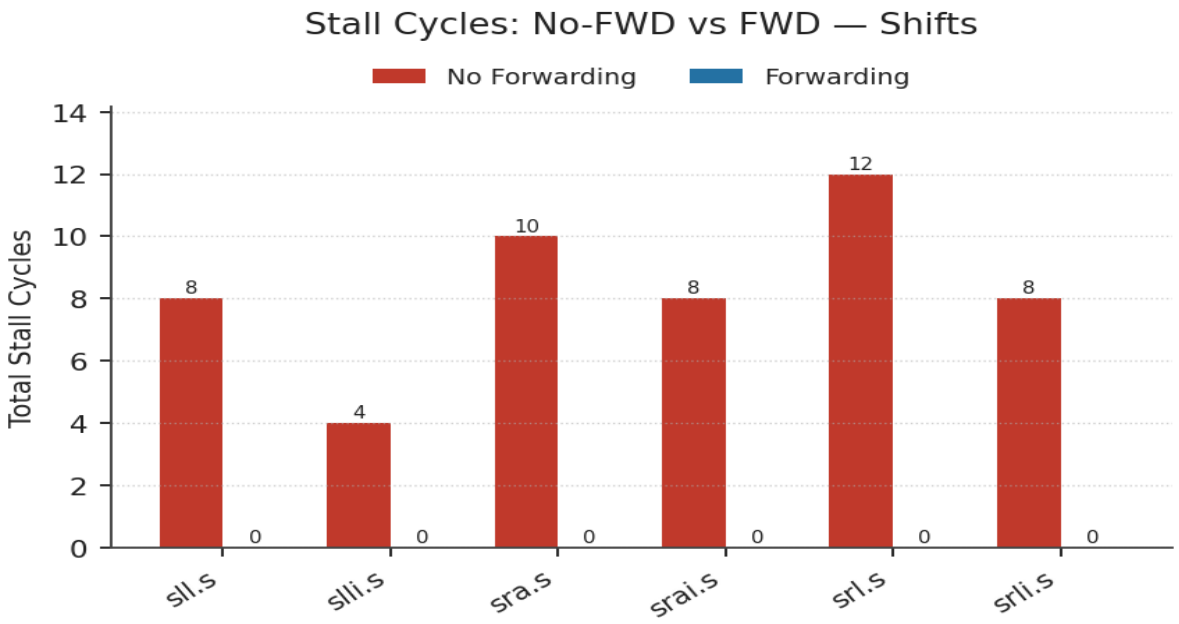
ALU / Logic / Compare Instructions



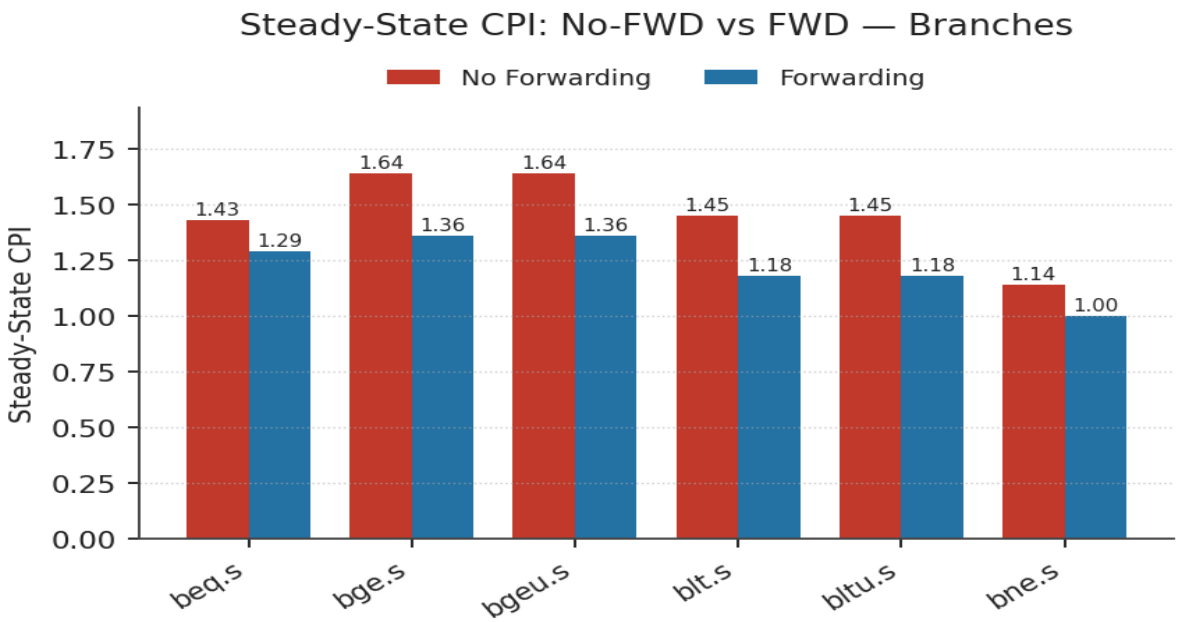


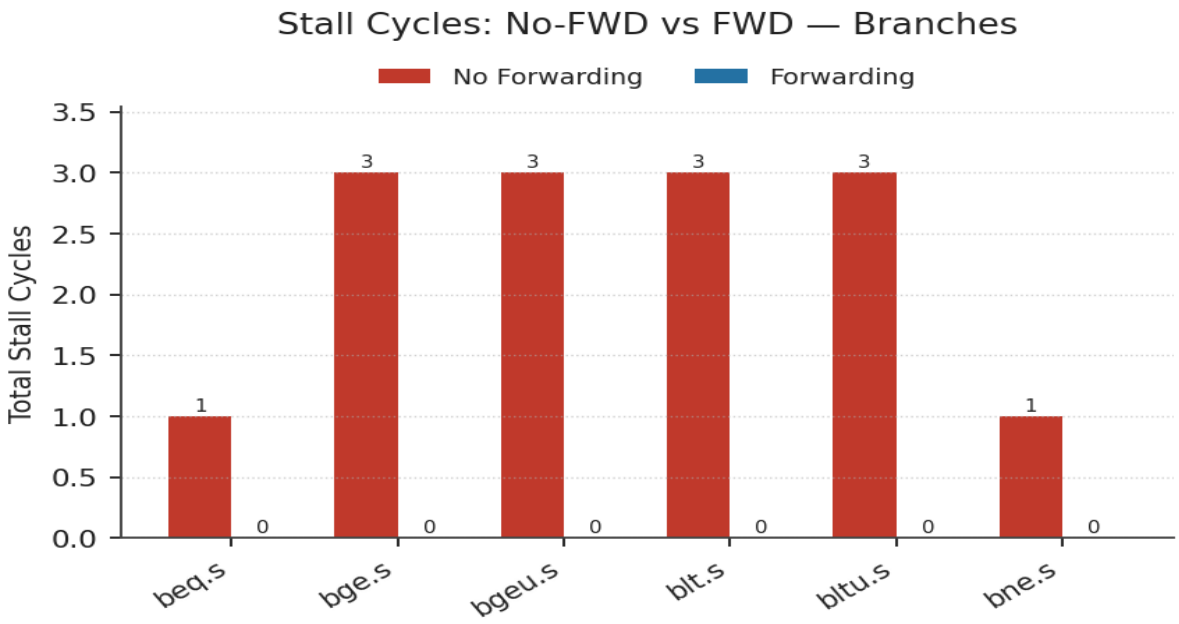
Shift Instructions



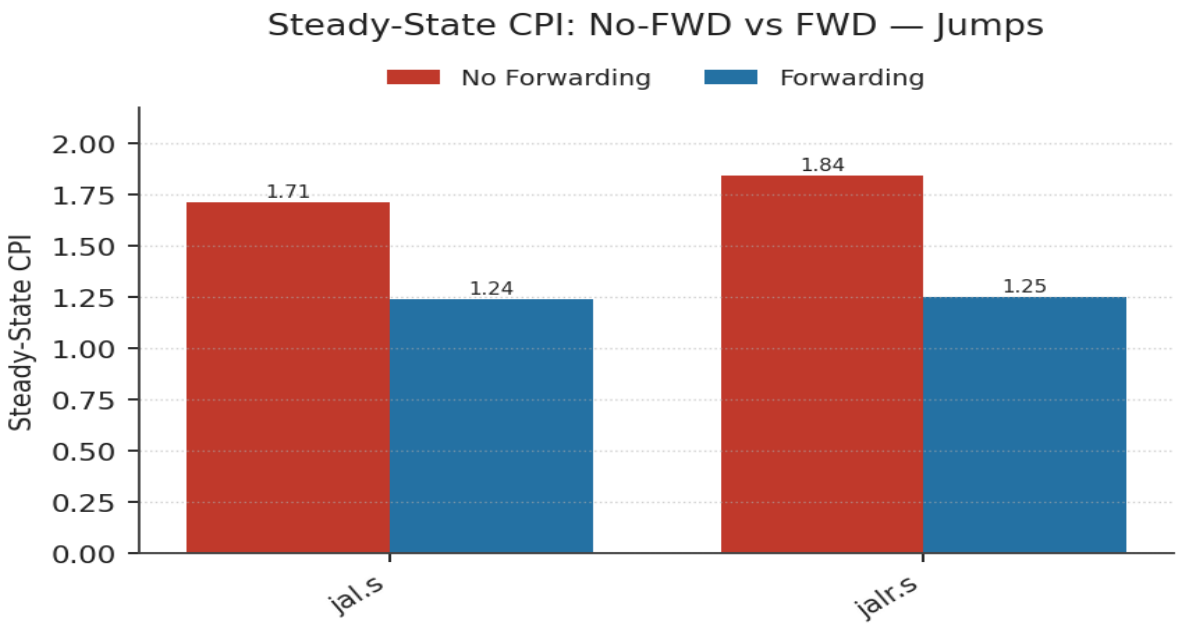


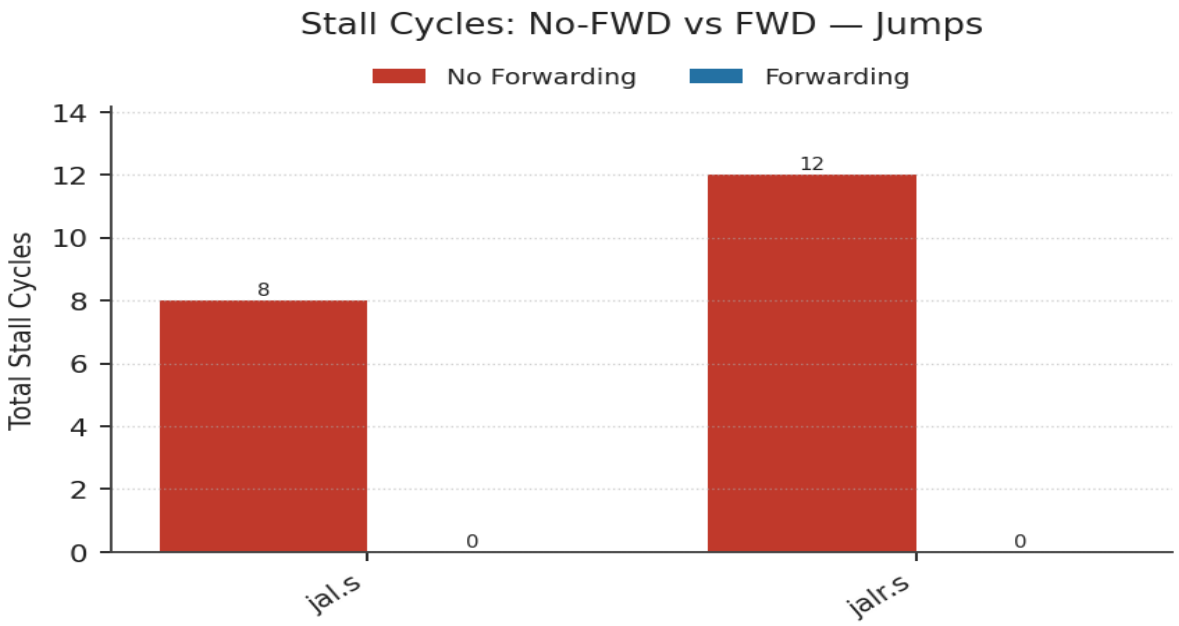
Branch Instructions



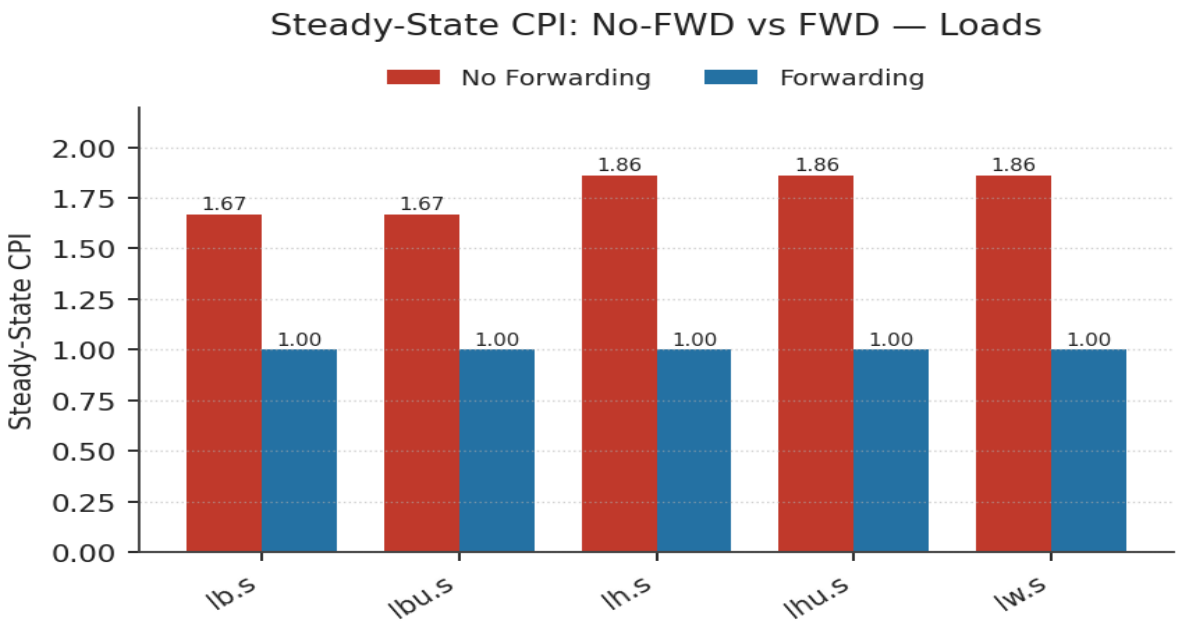


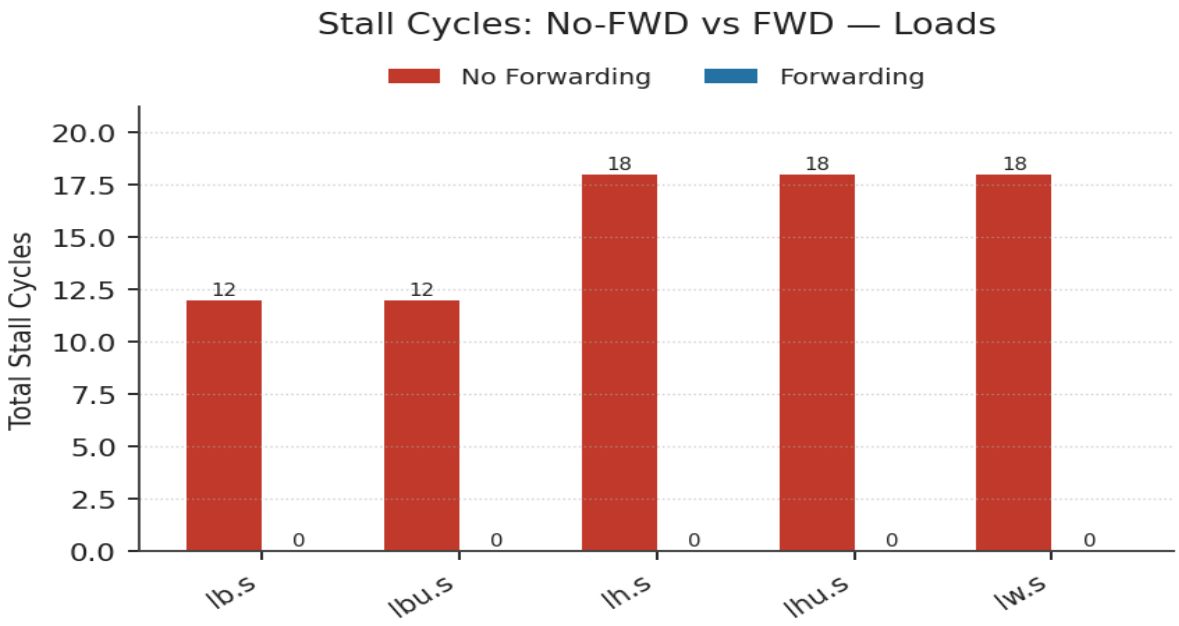
Jump Instructions



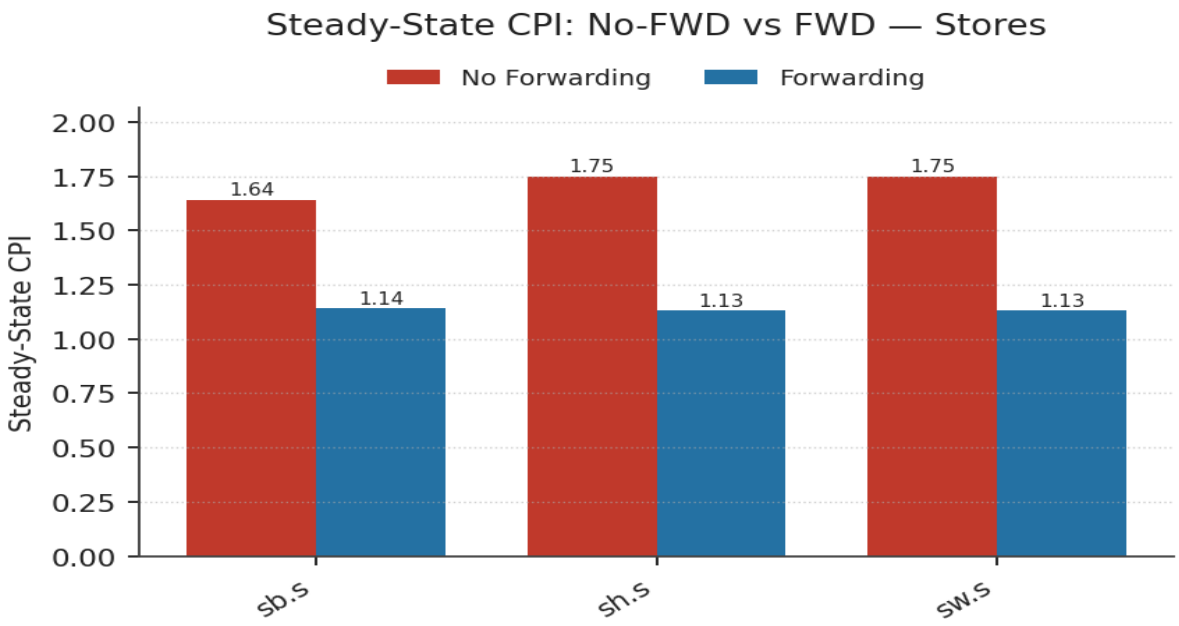


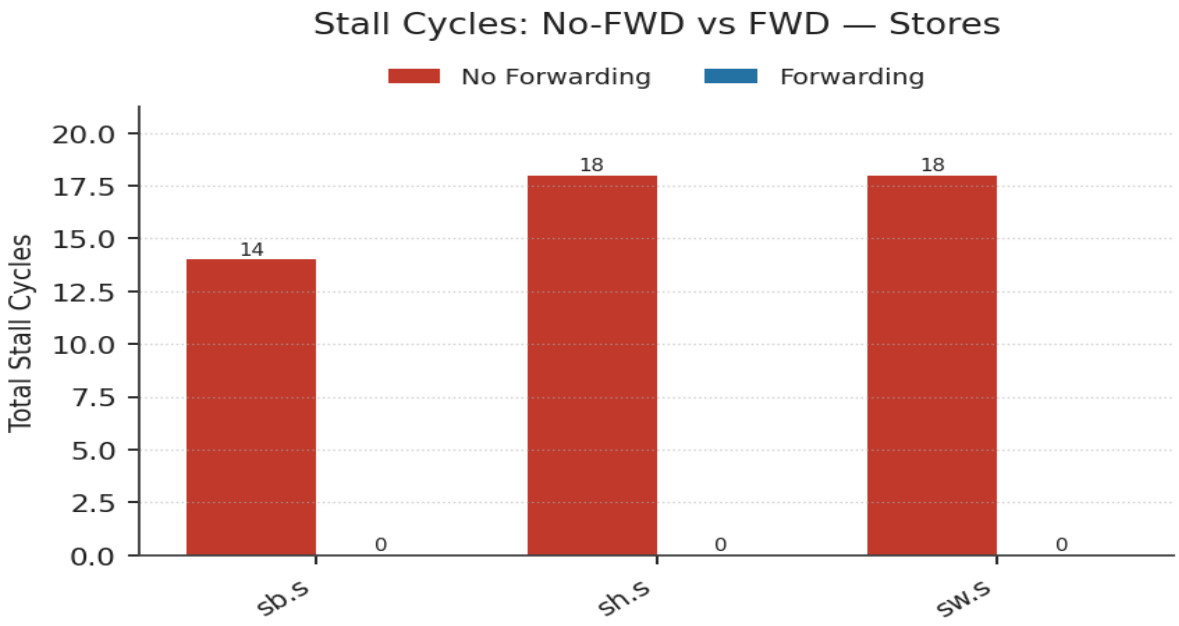
Load Instructions





Store Instructions





Dedicated Hazard-Suite Tests

Three tests were written specifically to stress the hazard-handling logic rather than a single instruction: **no_hazard.s** (back-to-back independent instructions, a control for measuring zero-hazard baseline CPI), **data_hazard.s** (a dense chain of RAW register dependencies), and **branch_hazard.s** (a sequence of taken/not-taken branches back to back). Results confirm the expected picture: with no hazards present, the two configurations are identical; with a dense dependency chain, forwarding removes essentially all stalling; and since branch resolution is unrelated to forwarding, branch_hazard.s shows no change in cycle count or CPI between configurations (its flush cycles are intrinsic to branch resolution timing, not data forwarding).

Test	Config	Cycles	Instr. Retired	Measured CPI	Steady-State CPI	Stall Cycles	Flush Cycles
no_hazard.s	No Forwarding	40	36	1.11	1.00	0	0
no_hazard.s	Forwarding	40	36	1.11	1.00	0	0
branch_hazard.s	No Forwarding	43	37	1.16	1.05	0	2
branch_hazard.s	Forwarding	43	37	1.16	1.05	0	2
data_hazard.s	No Forwarding	58	52	1.12	1.04	2	0
data_hazard.s	Forwarding	56	52	1.08	1.00	0	0

Table 1. Dedicated hazard-stress test results, both configurations.

Complete Per-Test Results

Full tabulated results for all 40 passing test programs, grouped by instruction category. The final two columns are the percentage improvement of the forwarding configuration over the no-forwarding baseline.

ALU / Logic / Compare Instructions

Test	Cyc NoFwd	Cyc Fwd	CPI NoFwd	CPI Fwd	Stall NoFwd	Stall Fwd	Flush NoFwd	Flush Fwd	CPI Impr.	Cyc Impr.
add.s	30	22	1.44	1.00	8	0	0	0	31%	27%
addi.s	27	19	1.53	1.00	8	0	0	0	35%	30%
and.s	19	15	1.36	1.00	4	0	0	0	26%	21%
andi.s	18	14	1.40	1.00	4	0	0	0	29%	22%
auipc.s	36	20	2.00	1.00	16	0	0	0	50%	44%
lui.s	31	17	2.08	1.00	14	0	0	0	52%	45%
or.s	27	19	1.53	1.00	8	0	0	0	35%	30%
ori.s	25	17	1.62	1.00	8	0	0	0	38%	32%
sub.s	28	20	1.50	1.00	8	0	0	0	33%	29%
xor.s	19	15	1.36	1.00	4	0	0	0	26%	21%
xori.s	18	14	1.40	1.00	4	0	0	0	29%	22%
slt.s	23	19	1.27	1.00	4	0	0	0	21%	17%
slti.s	21	17	1.31	1.00	4	0	0	0	24%	19%
sltiu.s	23	19	1.27	1.00	4	0	0	0	21%	17%
sltu.s	25	21	1.24	1.00	4	0	0	0	19%	16%

Shift Instructions

Test	Cyc NoFwd	Cyc Fwd	CPI NoFwd	CPI Fwd	Stall NoFwd	Stall Fwd	Flush NoFwd	Flush Fwd	CPI Impr.	Cyc Impr.
sll.s	27	19	1.53	1.00	8	0	0	0	35%	30%
slli.s	18	14	1.40	1.00	4	0	0	0	29%	22%
sra.s	31	21	1.59	1.00	10	0	0	0	37%	32%
srai.s	25	17	1.62	1.00	8	0	0	0	38%	32%
srl.s	37	25	1.57	1.00	12	0	0	0	36%	32%
srli.s	27	19	1.53	1.00	8	0	0	0	35%	30%

Branch Instructions

Test	Cyc NoFwd	Cyc Fwd	CPI NoFwd	CPI Fwd	Stall NoFwd	Stall Fwd	Flush NoFwd	Flush Fwd	CPI Impr.	Cyc Impr.
beq.s	14	13	1.43	1.29	1	0	2	2	10%	7%
bge.s	22	19	1.64	1.36	3	0	4	4	17%	14%
bgeu.s	22	19	1.64	1.36	3	0	4	4	17%	14%
blt.s	20	17	1.45	1.18	3	0	2	2	19%	15%
bltu.s	20	17	1.45	1.18	3	0	2	2	19%	15%
bne.s	12	11	1.14	1.00	1	0	0	0	12%	8%

Jump Instructions

Test	Cyc NoFwd	Cyc Fwd	CPI NoFwd	CPI Fwd	Stall NoFwd	Stall Fwd	Flush NoFwd	Flush Fwd	CPI Impr.	Cyc Impr.
jal.s	33	25	1.71	1.24	8	0	4	4	27%	24%
jalr.s	39	34	1.84	1.25	12	0	4	6	32%	13%

Load Instructions

Test	Cyc NoFwd	Cyc Fwd	CPI NoFwd	CPI Fwd	Stall NoFwd	Stall Fwd	Flush NoFwd	Flush Fwd	CPI Impr.	Cyc Impr.
lb.s	34	22	1.67	1.00	12	0	0	0	40%	35%
lbu.s	34	22	1.67	1.00	12	0	0	0	40%	35%
lh.s	43	25	1.86	1.00	18	0	0	0	46%	42%
lhu.s	43	25	1.86	1.00	18	0	0	0	46%	42%
lw.s	43	25	1.86	1.00	18	0	0	0	46%	42%

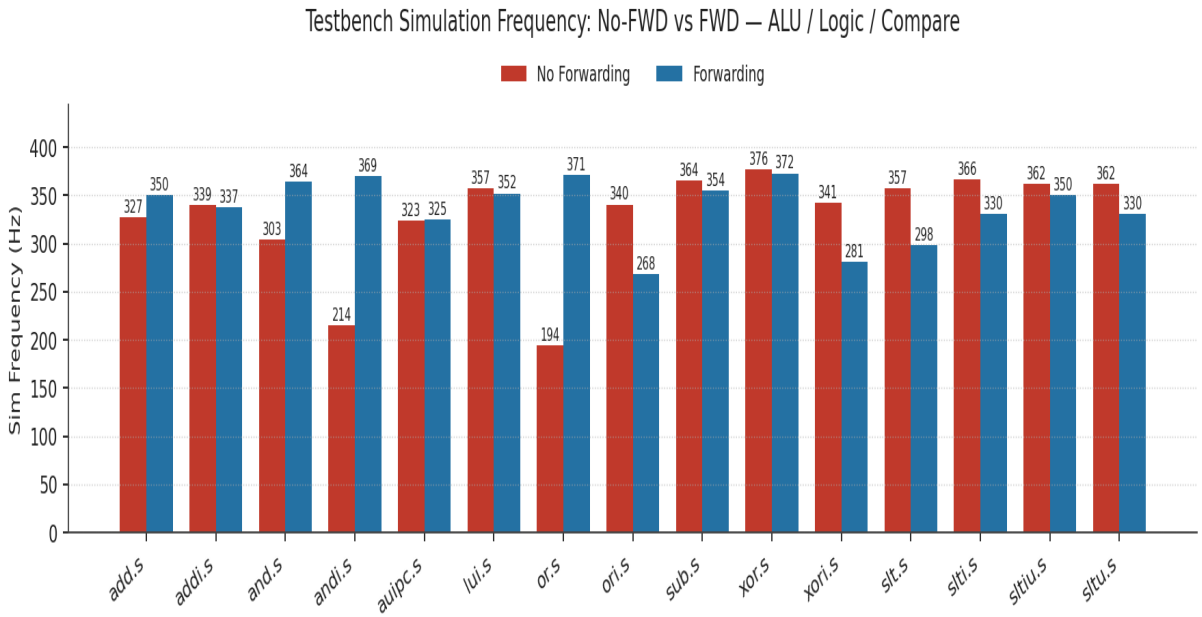
Store Instructions

Test	Cyc NoFwd	Cyc Fwd	CPI NoFwd	CPI Fwd	Stall NoFwd	Stall Fwd	Flush NoFwd	Flush Fwd	CPI Impr.	Cyc Impr.
sb.s	40	20	1.64	1.14	14	0	0	0	30%	50%
sh.s	46	21	1.75	1.13	18	0	0	0	35%	54%
sw.s	46	21	1.75	1.13	18	0	0	0	35%	54%

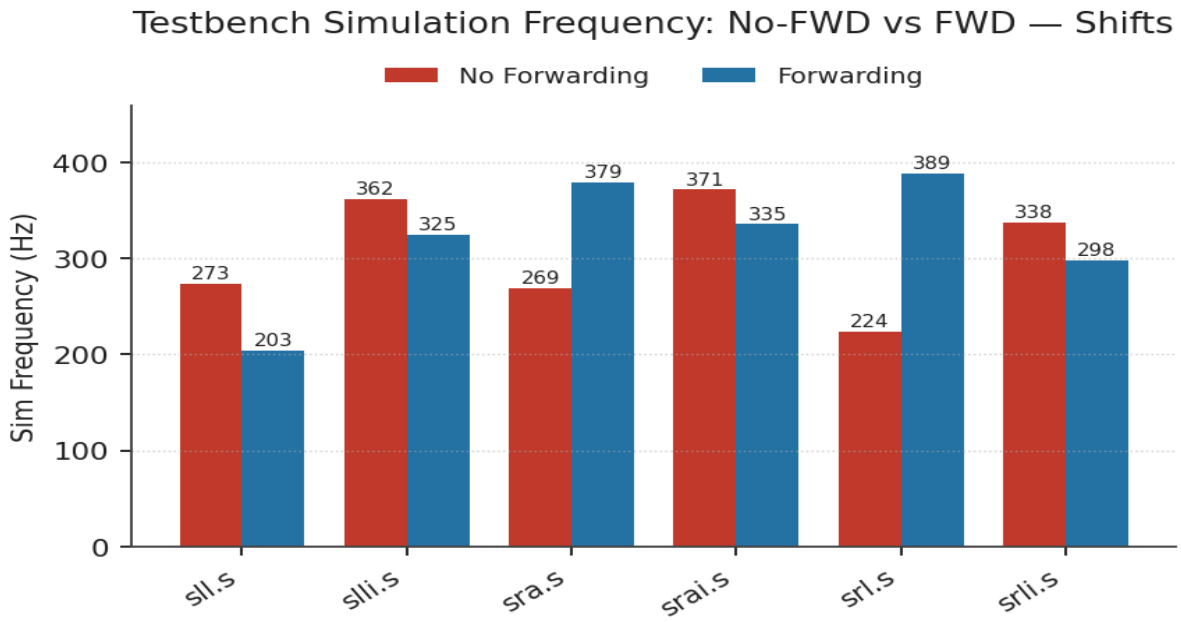
Appendix: Testbench Simulation Frequency

Simulation frequency measures host simulator throughput (cycles of RTL simulated per second of wall-clock host time) — a property of the simulation environment and test length, not of the synthesized core's timing. It is included here for completeness and visibility into testbench run-to-run variance, not as a performance claim about the core itself.

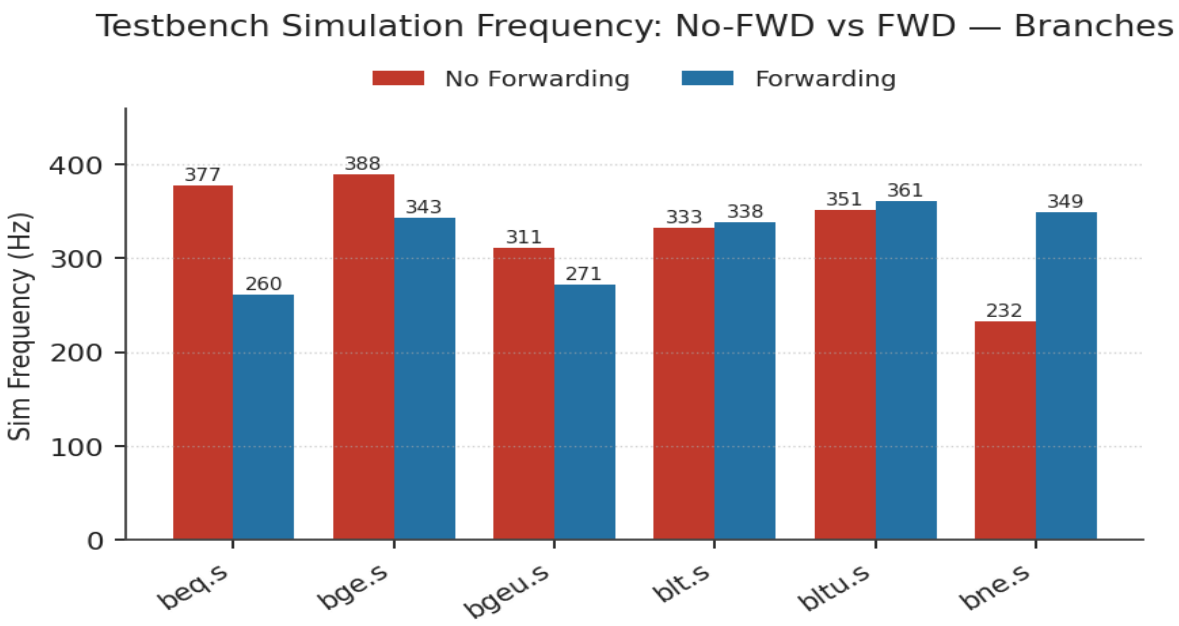
ALU / Logic / Compare Instructions



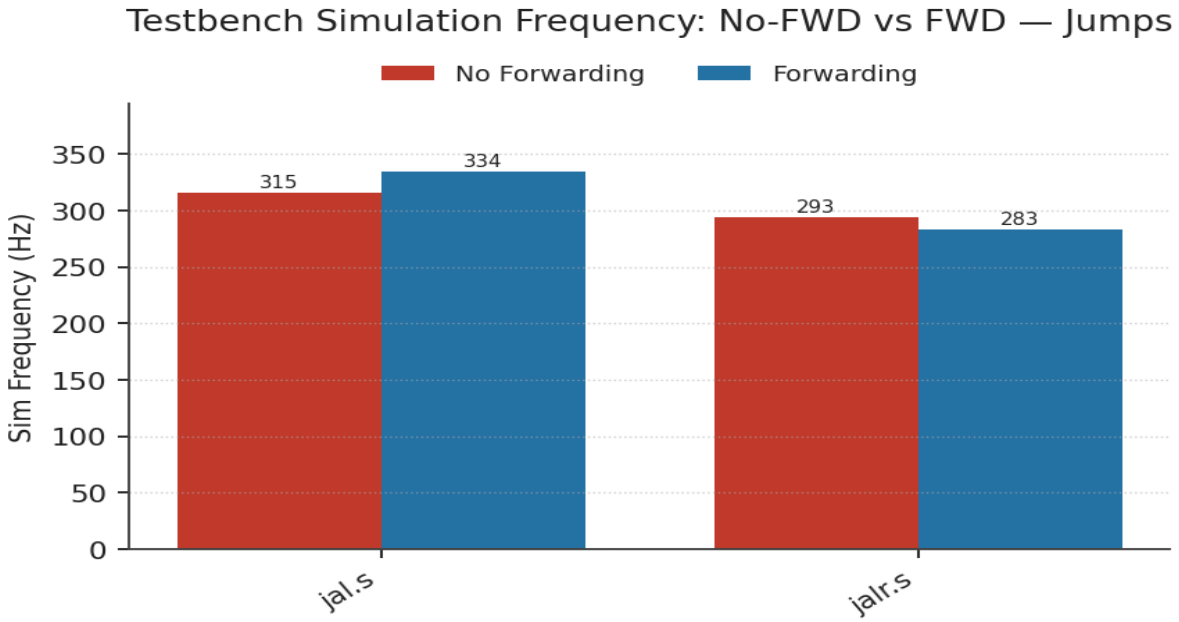
Shift Instructions



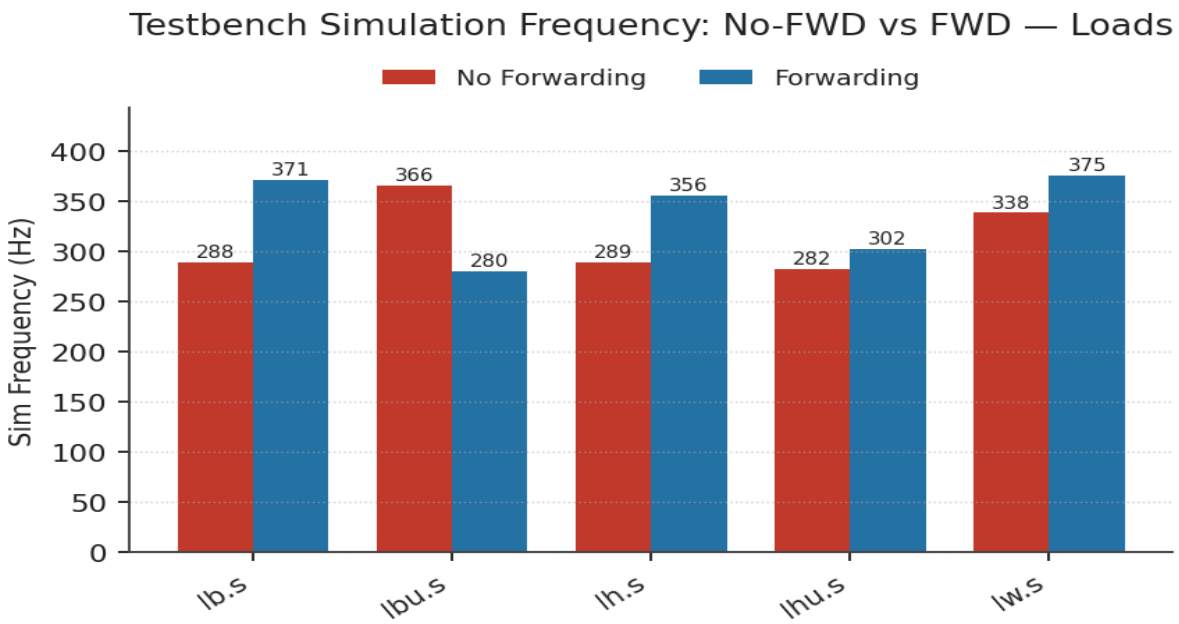
Branch Instructions



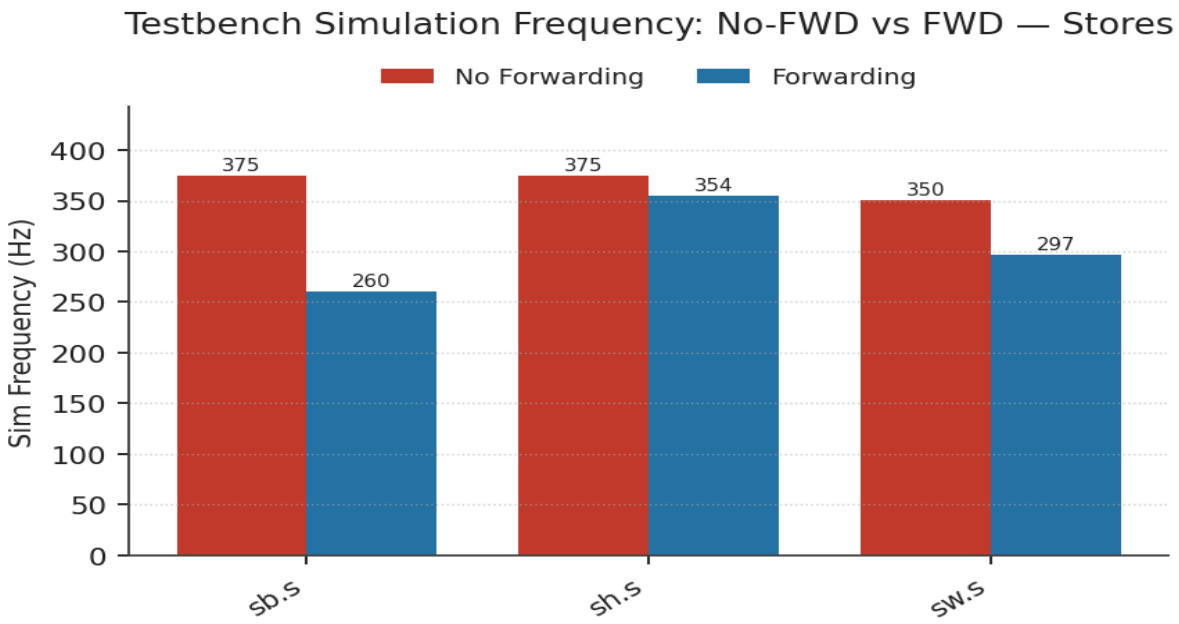
Jump Instructions



Load Instructions



Store Instructions



End of report — Zenith Core pipelined RV32I verification suite.